A Course End Project Report on

AI-Powered Code Debugger

A9503 - Data Structures Laboratory

Submitted by

Kunduru Bharath Reddy	25881A05CL
Gubba Saketh	25881A05DX
Appidi Vivek Reddy	25881A05EP
Mogili Kruthika	25881A05DC
Ramavath Anjali	25881A05CF

Course Facilitator

Mr. Ganesh Deshmukh
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

Academic Year 2025–2026 “‘

Contents

ABSTRACT

The AI-Powered Code Debugger is a software application developed using the C++ programming language to assist programmers in identifying, analyzing, and resolving errors in source code efficiently. The project combines static code analysis, compilation-based error detection, logging mechanisms, and AI-assisted debugging suggestions into a single integrated platform. The primary objective of the system is to reduce the time and effort required during the debugging process while improving code quality and developer productivity.

The system consists of multiple modules including a Menu Module, Static Analyzer, Compiler Module, AI Module, and Logger Module. The Static Analyzer scans source code files and detects common syntax-related issues such as missing semicolons and braces. The Compiler Module validates the source code using the GCC compiler and generates detailed error reports. The AI Module is designed to provide intelligent debugging suggestions and can be integrated with modern Artificial Intelligence services such as Gemini API. The Logger Module records debugging activities and stores reports for future reference.

The project demonstrates the practical application of data structures, file handling, modular programming, and software engineering principles. By automating various stages of the debugging process, the system minimizes manual effort and enhances development efficiency. The modular architecture of the application improves maintainability and scalability, making it suitable for future enhancements such as automatic code correction, graphical user interfaces, and support for multiple programming languages.

Overall, the AI-Powered Code Debugger serves as an effective educational and practical tool for understanding modern debugging techniques while providing a foundation for advanced AI-assisted software development systems.

Chapter 1

INTRODUCTION

Software development is a complex process that involves designing, coding, testing, and maintaining applications. During development, programmers frequently encounter errors such as syntax mistakes, logical flaws, and structural issues. Debugging these errors manually can be time-consuming and may reduce development productivity. To address this challenge, the **AI-Powered Code Debugger** has been developed as an intelligent software solution that assists programmers in identifying and resolving coding errors efficiently.

The project combines traditional debugging techniques with modern Artificial Intelligence concepts to provide a faster and more effective debugging experience. The system performs static analysis, compilation-based error detection, logging, and AI-assisted debugging suggestions through a user-friendly menu-driven interface.

Key Features of the System

- **Static Code Analysis** The system scans source code files and identifies common syntax-related issues such as missing semicolons, unmatched braces, and formatting errors.
- **Compiler-Based Validation** The application uses the GCC compiler to compile source code and detect compilation errors, ensuring program correctness.
- **AI-Assisted Suggestions** An AI module is integrated into the system to provide intelligent debugging recommendations and error explanations. The module can be extended using APIs such as Gemini AI.
- **Logging and Reporting** All debugging activities, compiler outputs, and analysis reports are stored for future reference and documentation purposes.
- **Menu-Driven Interface** The application provides a simple and interactive console-based menu that allows users to easily access different debugging functionalities.
- **Modular Architecture** The project is divided into independent modules such as Menu Module, Analyzer Module, Compiler Module, AI Module, and Logger Module, improving maintainability and scalability.

Importance of the Project

- Reduces the time required to identify programming errors.
- Improves software quality and code reliability.
- Helps beginners understand compiler errors more effectively.
- Demonstrates practical implementation of Data Structures and C++ programming concepts.
- Provides a foundation for future AI-powered software development tools.

The AI-Powered Code Debugger serves as both an educational and practical software tool. It demonstrates the application of data structures, file handling, modular programming, software engineering principles, and artificial intelligence concepts while addressing real-world debugging challenges. The project can be further enhanced by incorporating automatic code correction, graphical user interfaces, cloud-based debugging services, and support for multiple programming languages.

Chapter 2

OBJECTIVES

The AI-Powered Code Debugger is designed to simplify and enhance the software debugging process by combining traditional debugging techniques with modern Artificial Intelligence concepts. The project aims to assist programmers in identifying, analyzing, and resolving errors efficiently while improving overall software quality and development productivity.

The primary objectives of the project are as follows:

- **To Develop an Intelligent Debugging System** The main objective of the project is to design and implement a software application capable of assisting programmers in detecting and resolving coding errors automatically.
- **To Perform Static Code Analysis** The system should analyze source code files without executing them and identify common syntax-related issues such as missing semicolons, unmatched braces, and improper code structures.
- **To Integrate Compiler-Based Validation** The project aims to utilize the GCC compiler to verify source code correctness and generate detailed compilation error reports for debugging purposes.
- **To Provide AI-Assisted Debugging Suggestions** An Artificial Intelligence module is incorporated to provide intelligent recommendations and explanations for programming errors, making debugging easier for developers and students.
- **To Generate Detailed Error Reports** The application should generate comprehensive debugging reports containing analysis results, compiler messages, and suggested corrections.
- **To Maintain Debugging Logs** The system should store debugging activities and execution logs for future reference, analysis, and documentation.
- **To Reduce Debugging Time** By automating error detection and analysis, the project aims to significantly reduce the time required to locate and fix programming errors.
- **To Improve Software Quality** The system encourages developers to write cleaner and more reliable code by identifying mistakes during the development phase.

- **To Enhance Developer Productivity** The project assists programmers in focusing on problem-solving and application development rather than spending excessive time on manual debugging.
- **To Demonstrate Data Structures Concepts** The implementation of the project provides practical exposure to data structures, file handling, modular programming, and software engineering principles.
- **To Create a User-Friendly Interface** The application provides a menu-driven interface that enables users to interact with different debugging functionalities in a simple and efficient manner.
- **To Support Learning and Education** The project serves as an educational tool for students and beginner programmers by helping them understand compiler errors and debugging techniques.
- **To Build a Scalable Software Architecture** The modular design of the system allows future enhancements and easy integration of additional functionalities without affecting existing components.
- **To Provide a Foundation for Future Research** The project establishes a base for future developments such as automatic code correction, cloud-based debugging systems, graphical user interfaces, and support for multiple programming languages.

The successful achievement of these objectives results in a reliable, efficient, and intelligent debugging platform capable of improving the software development process while demonstrating the practical application of modern programming and Artificial Intelligence technologies.

Chapter 3

METHODOLOGY

3.1 System Architecture

The AI-Powered Code Debugger is designed using a modular software architecture that divides the entire system into multiple independent components. Each module is responsible for a specific functionality and interacts with other modules to achieve the overall objective of efficient code debugging. The modular design improves maintainability, scalability, readability, and ease of future enhancements.

The system consists of the following major modules:

1. **Menu Module**

   The Menu Module serves as the primary user interface of the application. It allows users to interact with different functionalities of the debugger through a menu-driven environment. The module manages user inputs and directs program execution to the appropriate subsystem.

2. **Static Analyzer Module**

   The Static Analyzer Module examines source code files without executing them. It scans the code and identifies common syntax-related issues such as:

   - Missing semicolons
   - Unmatched braces
   - Incorrect code formatting
   - Incomplete statements
   - Structural inconsistencies

   This module helps programmers identify errors at an early stage of development.

3. **Compiler Module**

   The Compiler Module utilizes the GCC compiler to compile source code files and verify their correctness. If compilation errors are encountered, the module captures and stores detailed compiler messages. This assists programmers in understanding the exact cause of errors and improves debugging efficiency.

4. **AI Suggestion Module**

   The AI Module is responsible for generating intelligent debugging suggestions. It analyzes source code and compiler messages to provide recommendations for resolving errors. The module is designed to support integration with advanced Artificial Intelligence services such as Gemini API, allowing future enhancements in automated code analysis and correction.

5. **Logger Module**

   The Logger Module records debugging activities, compiler outputs, and generated reports. It maintains logs for future reference and enables users to track debugging sessions effectively.

The interaction among these modules creates a complete debugging workflow capable of detecting errors, validating source code, generating suggestions, and maintaining execution records.

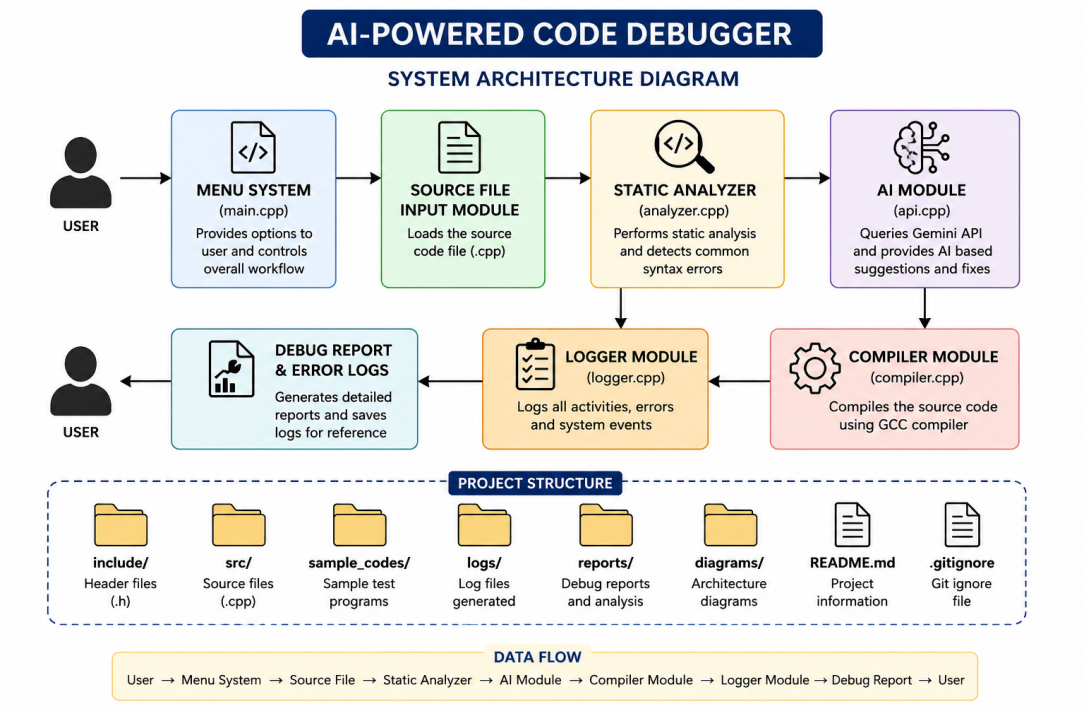

Figure 3.1: System Architecture of AI-Powered Code Debugger

3.2 Hardware Requirements

The hardware requirements for the AI-Powered Code Debugger are minimal, making the system suitable for deployment on standard desktop and laptop computers. The application does not require specialized hardware resources and can operate efficiently on commonly available computing systems.

- **Processor:** Intel Core i3 or higher

  A modern multi-core processor ensures efficient execution of source code analysis, compilation tasks, and report generation activities.

- **RAM:** Minimum 4 GB

  Sufficient memory is required to load source files, execute the application, and process compiler outputs efficiently.

- **Storage:** Minimum 20 GB Free Space

  Storage is necessary for source code files, debugging reports, log files, compiler outputs, and project documentation.

- **Internet Connection:**

  An internet connection is recommended for future AI integration using cloud-based services such as Gemini API.

- **Display:**

  A standard monitor with a minimum resolution of 1366×768 is sufficient for development and execution purposes.

3.3 Software Requirements

The project is developed using modern software development tools and technologies. The software requirements are listed below:

- **Programming Language: C++**

  C++ is used as the primary programming language due to its object-oriented features, efficiency, and extensive support for software development.

- **Compiler: GCC (GNU Compiler Collection)**

  The GCC compiler is used to compile source code and identify syntax and compilation errors.

- **Integrated Development Environment (IDE): Visual Studio Code**

  Visual Studio Code provides source code editing, debugging support, terminal integration, and extension support for efficient project development.

- **Version Control System: Git**

  Git is used to manage source code changes and maintain project versions during development.

- **Repository Hosting Platform: GitHub**

  GitHub is used for collaborative development, project management, source code backup, and version tracking.

- **Operating System:**

  - Windows 10/11
  - Linux (Ubuntu/Fedora)

- **Artificial Intelligence Platform: Gemini API**

  The AI module is designed to support Gemini API integration for generating intelligent debugging suggestions and recommendations.

3.4 Working Principle

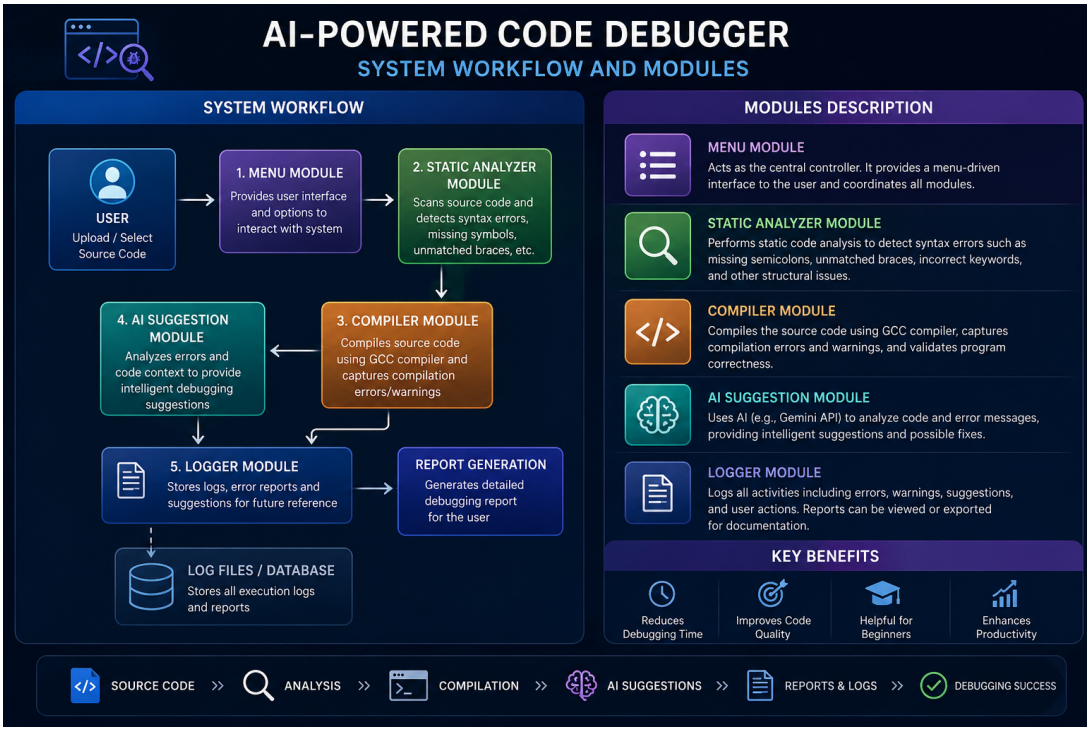

Figure 3.2: System Architecture of AI-Powered Code Debugger

The AI-Powered Code Debugger follows a systematic workflow for identifying and resolving programming errors.

1. The user launches the application and selects the required debugging operation from the menu-driven interface.
2. A source code file is provided as input to the system.
3. The Static Analyzer Module scans the source code and identifies common syntax-related issues such as missing semicolons, unmatched braces, and formatting errors.
4. The Compiler Module compiles the source code using GCC and records any compilation errors encountered during the process.
5. The generated compiler messages are analyzed and stored for debugging purposes.
6. The AI Suggestion Module examines the source code and compiler output to generate intelligent debugging recommendations.
7. The Logger Module records all activities, compiler outputs, and debugging suggestions into log files.
8. A debugging report is generated and presented to the user for review.
9. Based on the report and AI suggestions, the programmer modifies the source code and repeats the debugging cycle until all errors are resolved.

This workflow significantly reduces manual debugging effort and improves software development productivity.

3.5 Software Implementation

The implementation of the AI-Powered Code Debugger follows modular programming principles and object-oriented design concepts. Each module is implemented independently and communicates with other modules through well-defined interfaces.

The project source code is organized into separate files responsible for specific functionalities:

- **main.cpp**

  Acts as the entry point of the application and controls the overall execution flow.

- **menu.cpp**

  Implements the menu-driven interface and handles user interactions.

- **analyzer.cpp**

  Contains the implementation of static analysis algorithms and syntax error detection mechanisms.

- **compiler.cpp**

  Responsible for invoking the GCC compiler and capturing compilation results.

- **api.cpp**

  Implements the AI module and manages communication with AI services.

- **logger.cpp**

  Handles logging operations and report generation.

- **Header Files**

  Provide declarations and interfaces required by different modules.

The modular implementation improves code readability, simplifies maintenance, and enables future enhancements without affecting existing functionalities. The project structure also supports collaborative development through Git and GitHub, making it suitable for academic and real-world software engineering practices.

The implementation successfully demonstrates the practical application of Data Structures, file handling, modular programming, software engineering principles, compiler design concepts, and Artificial Intelligence integration within a single software system.

Chapter 4

RESULTS AND DISCUSSION

4.1 Program Execution Results

The AI-Powered Code Debugger was successfully developed and tested using multiple source code files containing different types of programming errors. The system was evaluated based on its ability to perform static analysis, compile source code, generate debugging reports, and provide AI-assisted suggestions.

4.2 Main Menu Interface

The application provides a menu-driven interface that allows users to access different debugging functionalities.

```
‘‘‘
  AI-Powered Code Debugger
‘‘‘

====================================

1. Load Source File
2. Run Static Analysis
3. Query Gemini AI
4. Compile Code
5. Generate Report
6. Exit
   ====================================
```

The menu interface ensures easy navigation and improves user interaction with the debugging system.

4.3 Static Analysis Results

The Static Analyzer Module successfully detected syntax-related issues in source code files without executing them.

Test Case: Missing Semicolons

Input File:

```
sample_codes/missing_semicolons.cpp
```

Output:

```
===== STATIC ANALYSIS REPORT =====
Line 7: Possible missing semicolon.
Line 10: Possible missing semicolon.
===== ANALYSIS COMPLETE =====
```

The analyzer accurately identified the lines containing potential syntax errors, reducing the effort required for manual code inspection.

4.4 Compilation Results

The Compiler Module was tested using both incorrect and correct source code files.

Compilation Failure Case

Input File:

```
sample_codes/missing_semicolons.cpp
```

Output:

```
Compilation Failed!
Check compile_errors.txt for details.
```

The compiler successfully detected syntax errors and generated detailed compiler messages for debugging purposes.

Compilation Success Case

Input File:

```
sample_codes/correct_code.cpp
```

Output:

```
Compilation Successful!
Executable file created successfully.
```

This confirms that the system correctly validates source code and generates executable files when no errors are present.

4.5 AI Module Results

The AI module was integrated with the project architecture and tested for debugging assistance.

Output:

```
===== GEMINI AI RESPONSE =====
Gemini API setup complete.
Request execution will be added next.
```

The module structure supports future integration with Gemini API for advanced debugging recommendations and code analysis.

4.6 Logging and Report Generation

The Logger Module records debugging activities and compiler outputs. Generated reports can be used for documentation and future reference.

Features of the logging system include:

- Recording compiler outputs.
- Storing debugging reports.
- Maintaining execution history.
- Supporting future analytics and performance tracking.

4.7 Performance Analysis

The developed system demonstrated the following capabilities:

- Successfully detected syntax-related programming errors.
- Reduced manual debugging effort through automated analysis.
- Generated meaningful compiler-based error reports.
- Provided a modular architecture for future AI integration.
- Improved debugging efficiency and code quality.
- Supported maintainable and scalable software development practices.

4.8 Discussion

The experimental results demonstrate that the AI-Powered Code Debugger effectively combines static analysis, compiler-based validation, and AI-assisted debugging concepts within a single platform. The Static Analyzer accurately detected common syntax errors, while the Compiler Module validated source code correctness through GCC compilation. The Logger Module maintained execution records, and the AI Module established the foundation for intelligent debugging assistance.

The project successfully achieved its objectives and demonstrated the practical application of Data Structures, file handling, modular programming, software engineering principles, and Artificial Intelligence concepts. The results confirm that the system can significantly improve the debugging process by reducing development time and enhancing software reliability.

Chapter 5

MAPPING OF PROGRAM OUTCOMES AND SDGS

5.1 Program Outcomes (POs) Mapping

The AI-Powered Code Debugger project contributes to the achievement of several Program Outcomes (POs) by applying theoretical concepts of Data Structures, Software Engineering, Artificial Intelligence, and Modern Development Tools to solve real-world debugging problems.

5.2 Contribution Towards Sustainable Development Goals (SDGs)

The AI-Powered Code Debugger contributes to the achievement of several Sustainable Development Goals (SDGs) by supporting education, technological innovation, and efficient software development practices.

SDG 4: Quality Education

- Assists students in understanding programming errors and debugging techniques.
- Provides an interactive learning platform for software development.
- Enhances practical knowledge of Data Structures and C++ programming.
- Supports self-learning through AI-assisted debugging suggestions.

SDG 8: Decent Work and Economic Growth

- Improves developer productivity by reducing debugging time.
- Encourages efficient software development practices.
- Supports creation of high-quality software systems.
- Enhances professional skills required in the software industry.

PO	Contribution of the Project
PO1	Application of engineering knowledge, programming principles, and data structures concepts for developing an intelligent debugging system.
PO2	Identification, analysis, and investigation of programming errors using static analysis and compiler-based validation techniques.
PO3	Design and development of an AI-assisted debugging platform capable of detecting, analyzing, and reporting software errors efficiently.
PO4	Conducting testing and validation of source code through systematic debugging procedures and compiler-generated feedback.
PO5	Usage of modern software development tools and technologies such as GCC Compiler, Visual Studio Code, Git, GitHub, and Gemini AI integration.
PO9	Encourages teamwork and collaborative software development through version control systems and project management practices.
PO10	Improves technical communication by generating debugging reports, logs, and documentation for software maintenance.
PO12	Promotes lifelong learning by integrating modern Artificial Intelligence technologies and continuously evolving software development practices.

Table 5.1: Program Outcomes Mapping for AI-Powered Code Debugger

SDG 9: Industry, Innovation and Infrastructure

- Promotes innovation through integration of Artificial Intelligence in software debugging.
- Encourages development of intelligent software engineering tools.
- Supports technological advancement in software development processes.
- Provides a scalable architecture for future AI-powered development platforms.

SDG 17: Partnerships for the Goals

- Encourages collaborative software development using Git and GitHub.
- Supports teamwork and knowledge sharing among developers.
- Facilitates coordinated project development and version management.

5.3 Overall Impact

The AI-Powered Code Debugger has a significant impact on both software development and technical education. The project successfully combines traditional debugging techniques with modern Artificial Intelligence concepts to create an efficient and user-friendly debugging platform. By automating error detection, code analysis, and report generation, the system reduces the time and effort required to identify and resolve programming issues.

From an educational perspective, the project serves as an effective learning tool for students and beginner programmers. It helps users understand compiler errors, syntax issues, and debugging methodologies in a systematic manner. The integration of AI-assisted suggestions further enhances the learning experience by providing explanations and recommendations that simplify the debugging process.

From a technological perspective, the project demonstrates the practical implementation of Data Structures, File Handling, Modular Programming, Compiler Concepts, Software Engineering Principles, and Artificial Intelligence integration. The use of modern development tools such as Visual Studio Code, GCC Compiler, Git, GitHub, and Gemini API highlights the adoption of industry-standard practices in software development.

The modular architecture of the system improves maintainability, scalability, and future extensibility. New functionalities such as automatic code correction, support for multiple programming languages, cloud-based debugging services, graphical user interfaces, and advanced AI-powered code analysis can be integrated without major modifications to the existing system.

The project also promotes collaborative software development practices through the use of version control systems and structured project management techniques. Team members can efficiently contribute to development, track changes, and maintain project consistency using Git and GitHub.

Overall, the AI-Powered Code Debugger successfully addresses real-world debugging challenges while contributing to educational advancement, technological innovation, and professional software engineering practices. The project demonstrates how Artificial Intelligence can be effectively combined with conventional debugging methods to improve software quality, developer productivity, and learning outcomes, making it a valuable contribution toward both Program Outcomes (POs) and Sustainable Development Goals (SDGs).

Chapter 6

CONCLUSION

The AI-Powered Code Debugger successfully demonstrates the integration of static code analysis, compiler-based validation, logging mechanisms, and AI-assisted debugging support into a single software application. The system efficiently identifies common programming errors, assists developers in debugging source code, and generates useful reports that improve software quality and development productivity. The project also showcases the practical implementation of Data Structures, File Handling, Software Engineering principles, and modern development tools.

Furthermore, the modular architecture of the application makes it scalable and easy to maintain, allowing future enhancements such as automatic code correction, advanced AI-based code analysis, graphical user interfaces, and support for multiple programming languages. Overall, the project serves as a valuable educational and practical tool that bridges traditional debugging techniques with modern Artificial Intelligence technologies, contributing to more efficient and intelligent software development practices.